Using Data Structures

(https://github.com/learn-co-curriculum/python-p3-dsa-using-data-structures) (https://github.com/learn-co-curriculum/python-p3-dsa-using-data-structures/issues/new)

Learning Goals

- Define "Data Structure".
- Evaluate the properties of common data structures (lists and dictionaries).
- Establish a general process to build data structures.
- Identify when to use different data structures.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO)
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

What is a data structure? In essence, a data structure is a tool for organizing a collection data so that we can interact with it efficiently. All data structures share the following characteristics:

- They store **collections of values**.
- They also store the **relationship between those values**.
- They provide **methods for interacting with those values**.

Consider a data structure you've been using for quite some time: a **list**. Lists are a common data structure that are built in to most programming languages. In Python, lists have the following characteristics:

- They stores a collection of values of any data type.
- They stores those values in an indexed sequence.
- They provide many methods for interacting with those values, like:
  - Accessing elements at a particular index position.
  - Adding elements.
  - Removing elements.
  - Iterating through every element.

Why Do We Need Different Data Structures?

While built-in data structures like Lists and Dictionaries are useful in many scenarios, it's also beneficial to be able to create custom data structures that can be used to help solve specific problems **more efficiently** than using built-in data structures. Different data structures excel in different situations.

For example, imagine a problem where you needed to add and remove elements from the beginning of a list. If we tried solving this problem using a list, our solution wouldn't be particularly efficient, since adding/removing elements from the beginning of an list has a Big O runtime of O(n), because adding/removing elements from the beginning of an list causes every other element in the list to be re-indexed. We can solve these kinds of problems using another data structure that you'll learn about later in this section, a **linked list**, which has a O(1) runtime for adding/removing elements from the beginning of the list.

In fact, you've already interacted a lot of different data structures besides just Lists and Dictionaries!

For example, any time you've interacted with the DOM in JavaScript, you've been interacting with a special data structure known as a **tree**; and `querySelector` (https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector) is a **tree traversal method** for efficiently finding a child element within the DOM tree.

React also uses its own custom data structure, the **React fiber tree** (https://github.com/acdlite/React-fiber-architecture#what-is-a-fiber), which allows it to efficiently perform updates to the DOM based on changes to state (and also is the reason you need to provide a special `key` prop for lists of elements).

Another place you've seen a common data structure in action is the **call stack** (https://en.wikipedia.org/wiki/Call_stack), which uses a **stack** data structure to keep track of all the currently running functions in a program and make sure they're executed in the correct order.

In the rest of lessons in this section, we'll show you how to build some of these common data structures, including linked lists, stacks, and binary search trees, and identify when to use them to solve specific problems.

Building Custom Data Structures

To get a sense of what the general process of building data structures is like, let's build out our very own version of the `list` data structure in Python. As a reminder, every data structure we make needs a way to do the following things:

- Store **collections of values**.
- Store the **relationship between those values**.
- Provide **methods for interacting with those values**.

To start off, every data structure we'll build will be defined as a class. For our array implementation lets call the class `MyArray`:

```
class MyArray:
  pass
```

Each data structure will also use **another data structure** in order to store the collection of values and help establish the relationships between them. To build our custom `MyArray` class, let's use a `dictionary` as the underlying data structure, along with a `length` attribute to keep track

of how many elements are in the array:

```
class MyArray:

  def __init__(self):
    self.dictionary = {}
    self.length = 0
```

Our data structures also need to provide **methods** for interacting with the data. Here's how we could implement `push` and `pop` for this class:

```
class MyArray:

  def __init__(self):
    self.dictionary = {}
    self.length = 0

  def push(self, value):
    self.dictionary[self.length] = value
    self.length += 1

  def pop(self):
    if self.length == 0:
        return None
    self.length -= 1
    return self.dictionary.pop(self.length)


arr = MyArray()
arr.push(1)
arr.push(2)
print(arr.pop())
```

```
# => 2
```

We've now defined our very own custom data structure! While it's highly unlikely that you'll need to build your own *array* class in the future, understanding this general approach to building data structures will help when you need to create other data structures that aren't provided by your programming language, such as a linked list.

Another advantage of building custom data structures like this for solving algorithm problems is that it makes it easier to understand the Big O runtime.

For example, in the code above, we can safely say that the `push` and `pop` operations have a O(1) runtime, since accessing and deleting elements from a Dictionary has a O(1) runtime. You can refer to this **Big O Cheat Sheet (https://www.bigocheatsheet.com/)** for more details on common runtimes.

> **If you'd like to explore this further, try adding methods to the MyArray class that add and remove elements from the beginning of MyArray. How does adding/removing elements from the beginning instead of the end of the list affect the runtime of those methods?**

Conclusion

In this lesson, we defined a "data structure" as a tool that stores a **collection of values** along with the **relationship between those values** and provides **methods for interacting with those values**.

Different data structures are more efficient at solving different problems, so the more data structures you familiarize yourself with, the more tools you'll have at your disposal to tackle different kinds of problems as efficiently as possible.

In general, building a data structure involves creating a class definition; using an auxiliary data structure to store values; and writing different methods for interacting with those values.

In the coming lessons, we'll explore several common data structures by providing instructions on how to build them, and then giving problems to practice using those data structures in different scenarios.

Resources

- **Big O Cheat Sheet (https://www.bigocheatsheet.com/)**